

PyNumDiff: Practical Numerical Differentiation for Noisy Data

Pavel Komarov¹, Floris van Breugel², Maria Protogerou⁴, and J. Nathan Kutz³

¹ Department of Electrical and Computer Engineering, University of Washington, USA ² Department of Mechanical Engineering, University of Nevada, Reno, USA ³ Autodesk Research, London, UK ⁴ Department of Applied Mathematics, University of Washington, USA ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Prashant Jha](#)

Reviewers:

- [@alexzerg](#)
- [@sidxz](#)

Submitted: 02 April 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

Derivatives of measured data are a prerequisite across science and engineering: identifying governing equations, designing controllers, and processing sensor streams alike. The textbook remedy, finite differencing, amplifies noise as $1/\Delta t$ and deteriorates rapidly as data grows noisier or more finely sampled. Smoothing before differencing helps, but algorithm choice and tuning substantially affect the result, and no single approach wins universally.

PyNumDiff is an open-source Python package consolidating a broad suite of numerical differentiation methods under a unified API. Seven algorithm families are implemented: (1) prefiltering followed by finite difference calculation; (2) iterated finite differencing; (3) polynomial fitting ([Savitzky & Golay, 1964](#)); (4) spectral, radial basis function, and wavelet fitting; (5) total variation regularization ([Chartrand, 2011](#)); (6) Kalman smoothing ([Kalman, 1960](#); [Rauch et al., 1965](#)); and (7) local approximation with linear models. Ahnert & Abel (2007) provides a useful taxonomy of these families, distinguishing local methods (which estimate derivatives from a surrounding window) from global methods that fit the entire signal at once. All PyNumDiff methods return a matched pair ($\hat{x}$, $\hat{dx/dt}$). A companion paper ([Komarov et al., 2025](#)) benchmarks all methods across test signals and guides selection for different application scenarios. This paper describes the package's second generation, the 0.2.x release series.

Statement of Need

Numerical differentiation has a diverse ecosystem of specialized algorithms, each with different strengths in outlier robustness, computational cost, irregular sampling, or missing observations, but no consolidated home. Without one, practitioners are left assembling solutions piecemeal from disparate packages ([van Breugel et al., 2022](#)).

PyNumDiff addresses this gap. Its unified interface lets users compare methods on the same data, exploit specialized capabilities, and select hyperparameters without ground-truth derivatives. Derivative estimation trades data fidelity against smoothness, and PyNumDiff frames that tradeoff as a multi-objective optimization spanning all methods, with a single scalar weight steering between the two. When a ground-truth derivative is available, it can serve as the optimization target instead ([van Breugel et al., 2020](#)). A natural and growing application is SINDy ([Brunton et al., 2016](#)), which discovers governing equations by regressing measured derivatives, making the quality of those estimates a direct determinant of model accuracy.

State of the Field

Relevant Python tools exist, but none covers PyNumDiff's breadth. `numpy.gradient` and `scipy.signal.savgol_filter` (Virtanen et al., 2020) handle only a sliver of the method space; `findiff` offers high-order finite difference stencils suited to clean simulation data, not noisy measurements. Historically, practitioners have had to stitch together PyKalman, PyDMD, and standalone TVR scripts (Chartrand, 2011) with no shared API or principled way to compare results. The derivative package (Goldschmidt, 2021) overlaps substantially but lacks multidimensional support, NaN handling, and hyperparameter optimization. No existing package spans PyNumDiff's seven method families with a consistent interface.

The original PyNumDiff publication (van Breugel et al., 2022) established the core method set and optimization framework. This second generation rewrites it from the ground up and consolidates it. `kerneldiff`, `rtstdiff`, and `tvrdiff` each cover by parameter what had been separate functions, so the package presents fewer entry points than before while spanning more methods. `rbfdiff`, `waveletdiff`, and `robustdiff` are new, bringing the set to the twelve methods of Table 1, all organized into the seven families above behind one keyword-argument signature. A single optimizer serves every one of them, and four capabilities now span the package, covering multidimensional data, irregular sample spacing, missing observations, and circular domains.

Software Design

Package design. All differentiation methods share the call signature

```
x_hat, dxdt_hat = method(x, dt_or_t, **params)
```

where `x` is a NumPy array (Harris et al., 2020) of measurements; `dt_or_t` is either a scalar step size or an array of sample locations; and keyword arguments configure the method, making calls self-documenting. Prior positional signatures are preserved with deprecation warnings.

Software architecture. PyNumDiff is organized into seven method modules plus shared `utils` and `optimize` modules in a flat structure. Where strong alternatives exist, PyNumDiff delegates rather than reimplements: SciPy (Virtanen et al., 2020) provides spline fitting, Savitzky-Golay filtering, and signal processing routines; NumPy (Harris et al., 2020) provides the FFT; PyWavelets (Lee et al., 2019) provides the discrete wavelet transform for `waveletdiff`; CVXPY (Diamond & Boyd, 2016) handles convex optimization for `robustdiff` and `tvrdiff` as an optional dependency. The public `kalman_filter` and `rts_smooth` primitives let users with known dynamics bypass `rtstdiff`'s constant-derivative model.

Method capabilities. Table 1 groups the twelve non-deprecated methods by capability. All handle multidimensional data via `axis`, and several go further.

Table 1: Methods by capability.

Capability	Methods
Multidimensional data	<code>kerneldiff</code> , <code>finitediff</code> , <code>polydiff</code> , <code>savgoldiff</code> , <code>splinediff</code> , <code>spectraldiff</code> , <code>rbfdiff</code> , <code>waveletdiff</code> , <code>tvrdiff</code> , <code>rtstdiff</code> , <code>robustdiff</code> , <code>lineardiff</code>
Variable step	<code>polydiff</code> , <code>splinediff</code> , <code>rbfdiff</code> , <code>rtstdiff</code> , <code>robustdiff</code>
Missing data	<code>polydiff</code> , <code>splinediff</code> , <code>rtstdiff</code> , <code>robustdiff</code>
Outlier robustness	<code>tvrdiff</code> , <code>robustdiff</code>
Circular domain	<code>rtstdiff</code>

Irregular and incomplete sampling. Methods that support variable step size accept an array of sample locations in place of a scalar step, and Kalman-based methods then compute the transition matrix by matrix exponential at each actual interval. NaN entries are treated as missing observations, excluded from fitting and imputed from the model, so sensors that drop samples need no preprocessing.

Outlier robustness. `robustdiff` replaces the quadratic Kalman cost with Huber loss terms on both measurement and process residuals, following Aravkin et al. (2013), with CVXPY (Diamond & Boyd, 2016) as the optimization backend; the sparse problem formulation scales linearly with signal length. `tvrdiff` similarly applies Huber loss on data fidelity; its total variation penalty on the derivative additionally promotes piecewise-smooth solutions for signals with abrupt transitions.

Circular and wrapped domains. `rtstdiff` accepts `circular=True` for quantities like angles on a periodic domain. Innovation residuals are wrapped to $[-\pi, \pi]$ before each Kalman update via an `innovation_fn` hook, and `x_hat` is returned in the same range, avoiding the large spurious spikes naive smoothers produce when a signal crosses the $\pm\pi$ boundary.

Hyperparameter optimization. `pynumdiff.optimize` selects hyperparameters Φ by minimizing

$$L(\Phi) = \text{RMSE}(\int \hat{x}(\Phi) + c, x) + \gamma \text{TV}(\hat{x}(\Phi)),$$

which requires no ground truth, because fidelity is measured by integrating the estimated derivative back against the measured signal (van Breugel et al., 2020). The smoothness weight γ , exposed as `tvgamma`, can be initialized automatically from the signal's estimated cutoff frequency. This version robustifies the first term with a Huber penalty so outliers do not bias parameter selection, and it reduces the Kalman parameter space from two independent noise variances to their log-ratio, the only salient factor (Komarov et al., 2025). Categorical and boolean hyperparameters are now supported natively, so discrete choices such as derivative order can be optimized jointly with continuous ones. Repeated evaluations are cached to avoid redundant work. A single call to `suggest_method` runs this search across nearly every method in the package and reports the best fit, which is the practical payoff of putting them all behind one interface.

Testing and continuous integration. The test suite validates all methods against analytic functions with known derivatives, covering both noiseless and noisy cases. Care was taken to avoid tautological tests where the implementation directly determines the expected result. GitHub Actions runs the suite on every push and pull request, and Coveralls tracks line coverage, currently 90%.

Research Impact Statement

The original PyNumDiff paper (van Breugel et al., 2022) has been applied in experimental biology (flight kinematics from motion capture), control engineering (observer design), and data-driven dynamics identification via SINDy (Brunton et al., 2016). Tutorial notebooks in the repository cover basic usage, hyperparameter optimization, automatic method suggestion, a head-to-head performance analysis across simulations, and demonstrations of the outlier, multidimensional, and circular-domain capabilities described above. Each runs end to end against the released package, so the claims made here can be reproduced directly. Full API documentation is published at pynumdiff.readthedocs.io. The companion Taxonomy paper (Komarov et al., 2025), submitted to the Journal of Computational Physics, provides the theoretical underpinning and benchmarks all included methods. The PySINDy project (Silva et al., 2020) maintains its own differentiation submodule substantially overlapping with PyNumDiff's capabilities; integration discussions are ongoing.

AI Usage Disclosure

This paper was drafted with assistance from Claude Sonnet 4.6 and Claude Opus 4.8 (Anthropic), which also implemented successive code revisions to address recent issues and author feedback. All outputs were reviewed and further edited by hand, and the authors take full responsibility for accuracy.

Acknowledgements

The authors thank Yuying Liu and Bingni W. Brunton for their contributions to the original PyNumDiff package (van Breugel et al., 2022), and Sasha Aravkin for discussions on convex optimization techniques that informed the robust differentiation methods. This work was supported by the NSF AI Institute in Dynamic Systems (grant number 2112085).

References

- Ahnert, K., & Abel, M. (2007). Numerical differentiation of experimental data: Local versus global methods. *Computer Physics Communications*, 177(10), 764–774. <https://doi.org/10.1016/j.cpc.2007.03.009>
- Aravkin, A. Y., Burke, J. V., & Pillonetto, G. (2013). Optimization viewpoint on Kalman smoothing with applications to robust and sparse estimation. *Journal of Machine Learning Research*, 14, 2513–2558. https://doi.org/10.1007/978-3-642-38398-4_8
- Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15), 3932–3937. <https://doi.org/10.1073/pnas.1517384113>
- Chartrand, R. (2011). Numerical differentiation of noisy, nonsmooth data. *ISRN Applied Mathematics*, 2011, 164564. <https://doi.org/10.5402/2011/164564>
- Diamond, S., & Boyd, S. (2016). CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83), 1–5.
- Goldschmidt, A. (2021). *Derivative: Numerical differentiation in Python*. <https://github.com/andgoldschmidt/derivative>
- Harris, C. R., Millman, K. J., van der Walt, S. J., & others. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1), 35–45. <https://doi.org/10.1115/1.3662552>
- Komarov, P., van Breugel, F., & Kutz, J. N. (2025). A taxonomy of numerical differentiation methods. <https://arxiv.org/abs/2512.09090>
- Lee, G. R., Gommers, R., Wasilewski, F., Wohlfahrt, K., & O’Leary, A. (2019). PyWavelets: A Python package for wavelet analysis. *Journal of Open Source Software*, 4(36), 1237. <https://doi.org/10.21105/joss.01237>
- Rauch, H. E., Tung, F., & Striebel, C. T. (1965). Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8), 1445–1450. <https://doi.org/10.2514/3.3166>
- Savitzky, A., & Golay, M. J. E. (1964). Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, 36(8), 1627–1639. <https://doi.org/10.1021/ac60214a047>
- Silva, B. M. de, Champion, K., Quade, M., Loiseau, J.-C., Kutz, J. N., & Brunton, S. L. (2020). PySINDy: A Python package for the sparse identification of nonlinear dynamics from data.

- 160 *Journal of Open Source Software*, 5(49), 2104. <https://doi.org/10.21105/joss.02104>
- 161 van Breugel, F., Kutz, J. N., & Brunton, B. W. (2020). Numerical differentiation of noisy
162 data: A unifying multi-objective optimization framework. *IEEE Access*, 8, 196865–196877.
163 <https://doi.org/10.1109/ACCESS.2020.3034077>
- 164 van Breugel, F., Liu, Y., Brunton, B. W., & Kutz, J. N. (2022). PyNumDiff: A Python
165 package for numerical differentiation of noisy time-series data. *Journal of Open Source*
166 *Software*, 7(71), 4078. <https://doi.org/10.21105/joss.04078>
- 167 Virtanen, P., Gommers, R., Oliphant, T. E., & others. (2020). SciPy 1.0: Fundamental
168 algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
169

DRAFT